

**농·림·수산식품부**

?S I 4 X I I - 2 p w / N I y y p - S s w p - p y # b O G x - p e l x ,  
2 ? Y O S U J 534>  
? 7 + , .  
? H ,

|            |        |      |       |                    |  |   |             |
|------------|--------|------|-------|--------------------|--|---|-------------|
| J 가 hv J H | YWRt p | URGV | /     | / X/               |  |   | tyO ↔ np—,  |
|            | 1      |      |       | I Ya/ VI ↔ p /PYUT |  | / |             |
|            | URGV   |      | 1K감가↔ | J 가 hv J H         |  |   | /J 가 hv J H |
|            |        |      | /     | 6< ㅆ               |  | 1 |             |
|            | /      |      |       | J 가 hv J HQaYY     |  | 1 |             |

D]  $\mathbb{A}^1$  a

$$+ 2X, \quad ?$$

분석 환경:

|                  |  |
|------------------|--|
| Jupyter Notebook |  |
| (또는 RStudio)     |  |
|                  |  |
| Python/R 코드      |  |
| → 데이터 처리         |  |
| → 머신러닝           |  |
| → 시각화            |  |

↓ 데이터 필요

- | 외부 데이터 소스
- | - CSV/JSON 파일
- | - 로컬 데이터베이스
- | - 클라우드 스토리지

?

41  
 $V \sim \frac{1}{p} YWR/S \frac{1}{p} YWR$        $\frac{1}{p}$       1  
+      1

51 401, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000

2

61  
k·꺾s~y  
% ?I Ya 를 JH 를 를 를 를 V꺾s~y

tx · ~↔↔ | yol | — o  
tx · ~↔↔ — 값~· r 5

%41l Ya  
oqC · o1p| oj n-각\*이 | h-각\*,

%51J H  
n~yy C · 값~· r 5h~yypn #omyl x pCx 값m가p+€· ~—+p-#

%61 + ,  
n가↔↔↔C n~yy h가↔↔↔+  
n가↔↔↔p값n가p# XKG\_K\_GHRK ol | +111#

%71  
q↔↔↔간ty oq1p↔↔↔간+?  
n가↔↔↔p값n가p#O YKX\_ σ\_U ol | aGR`KY +111#

%91  
n가↔↔↔p값n가p#YKRKI \_ 111LXUS ol | b NKXK 111#

%; 1  
↔p가w—C n가↔↔↔↔p hsl w↔  
ogj ↔p가wC · o1| | L↔x p+↔p가w—  
k

J가hvJ H ?  
k· 값5~y  
tx · ~↔↔o가hvom  
↔p가wC o가hvom1—w#YKRKI \_ - LXUS \*이 | h-각\*b NKXK 111# bq+  
k

71YWRt p  
YWRt p ↔↔↔간|—↔p URGV  
MXU`V Hd/PUσ/ 객

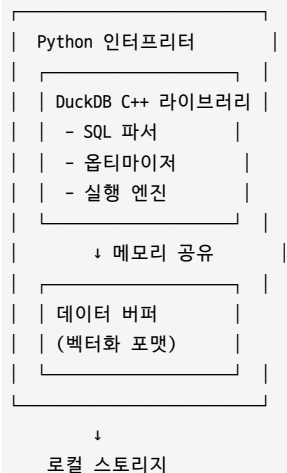
## 오상객p m

#YWRt p . V~—+↔pYWR URGV #  
J가hvJ H ?  
YWRt p / 를 고  
V~—+↔pYWR / /URGV 를

D] Æ a

D] Æ/D 4비값사무명사\$

호스트 프로세스 (Python/R)



(CSV, Parquet, etc)

?

4101

2

nl nsp x t

51

km-s

· t ty tw hvom

k

/

61

V 값 s ~ y

J H

## D] A E 4인 부분비값을 변경

Row-Store (전통적):

ID | Name | Age | Salary

-----

1 | Alice | 30 | 50000

2 | Bob | 25 | 45000

3 | Carol | 35 | 60000

메모리: [1,Alice,30,50000,2,Bob,25,45000,3,Carol,35,60000,...]

↑ 모든 데이터가 순서대로 저장

Column-Store (DuckDB):

ID: [1, 2, 3, ...]

Name: [Alice, Bob, Carol, ...]

Age: [30, 25, 35, ...]

Salary: [50000, 45000, 60000, ...]

메모리: [1,2,3,...] [Alice,Bob,Carol,...] [30,25,35,...] [50000,45000,60000,...]

URGV ?

41

k → w

YKRKI \_ yl x p/ + w 값 L XUS px · w 값 p b NKXK l rp D 63

k

l rp

- p w n L

+ yl x p/ + w 값

1

X ~ 값 10 ← p

1

51

I V`

Y S J + tyr w Q → h t y S 가 w t w J l t,

61

X ~ 값 10 ← p

## D] A E 4인 부분비값을 변경

전통적 인터프리터 실행:

| FOR each row in table:

| IF age > 30:

| output row

특성: 행 단위 처리, 함수 호출 오버헤드 크음

DuckDB 벡터화 실행:

```
| age_col = [22, 35, 40, 28, 50, ...] |
| mask = age_col > 30 # SIMD 연산 |
|   → [F, T, T, F, T, ...] |
| result = age_col[mask] |
|   → [35, 40, 50, ...] |
```

특성: 벡터 단위 처리, SIMD 최적화

?

433 / 433 / 4333

I V` ? m+yns · potn t~y,

px ~값 ← top, ?

Y S J ? I V` Gac0945

? Grp D 63 / 433 룰 433 /

1

D] AEO 4n사선로형가감분변명

J가vJH JH ?

SQL 쿼리

- ↓ (파싱)

추상 구문 트리 (AST)

- ↓ (검증)

논리 계획 (Logical Plan)

- ├ Filter(age > 30)
- ├ Project(name, salary)
- └ Scan(employees)

- ↓ (최적화)

물리 계획 (Physical Plan)

- ├ Seq. Scan + Filter (index 없으면)
- └ Index Scan (index 있으면)

- ↓ (실행)

결과

?

41 Ltwp←V가so~간y,

k

?

V↔upn tyl x p/←w←값

Ltwp←I rp D 63,

Ynl y+px · w2pp,

?

V↔upn tyl x p/←w←값

Ynl y+px · w2pp/I rp D 63, %

k

51 V↔upn t~y V가so~간y,

k

/

k

61 P~ty U←op~yr,

k

/

k

71 # I <otyl w 값 K-tx I t-y,

k

N 값 p<R~r R~r + I U`T\_ J Y\_σ I \_,

k

D] A ao 상백 m 관경

J 가 hvJ H J 가 hvJ H Qa YY ?

```
import duckdb

# 벡터 데이터 로드
conn = duckdb.connect(':memory:')
conn.execute("CREATE TABLE vectors (id INT, embedding FLOAT[1536])")

# HNSW 인덱스 생성
conn.execute("CREATE INDEX idx ON vectors USING HNSW (embedding)")

# 벡터 유사도 검색
result = conn.execute("""
    SELECT id,
           array_distance(embedding, [0.1, 0.2, ...]) AS dist
    FROM vectors
    ORDER BY dist
    LIMIT 10
""").fetchall()
```

?I <ot-lynp+ /I <ot~ j- <ogh t/ I <otn~tpj-tx tw< 값  
?NTYb + ,  
?QTT/ <y r p-pl <as  
?YWR 룰 YWR + /MXU`V Hd/ ,

J 가 hvJ H Qa YY ?  
NTYb + ,  
+ ,

D] A o 상백 m a

J 가 hvJ H 433( ?

```
# 예시: WHERE embedding <-> query_vec < 0.5
# DuckDB의 추정: "이 필터는 아무 효과 없음 (선택도 100%)"
# 실제: 선택도 10% ~ 50% (데이터에 따라 다름)
```

?

J 가 hvJ H ?  
2 ?b NKXK I r p D 63 룰  
?b NKXK yl x p C \*Gwp\* 룰 42t-lyn j-n~가 L  
?b NKXK px mpootyr BOD ?pn B s-p-s~w 룰 룰 433(  
/ + , 1

```
쿼리:
SELECT * FROM products
WHERE embedding <-> query_vec < 0.5
AND category = 'electronics'
```

J가hvJH ?

products 테이블: 100만 행  
벡터 필터 선택도: 100% (오류!)  
카테고리 필터 선택도: 5% (정확)  
결합 선택도: 100% \* 5% = 5% (잘못됨)  
추정 결과: 5만 행

?

벡터 필터 선택도: 20% (실제)  
카테고리 필터 선택도: 5% (정확)  
결합 선택도: 20% \* 5% = 1% (실제)  
실제 결과: 1만 행

? 9 / 1

D] R p 성능 선택 o 상환 m

K감가↔ J가hvJH ?

- 1. 벡터 필터 감지  
IF 쿼리에 embedding <-> vec < threshold 조건이 있으면:  
    벡터 필터로 태그 지정
- 2. ECQO 호출  
    ECQO 알고리즘 실행 → 정확한 선택도 계산
- 3. 옵티마이저 규칙 수정  
    논리 계획의 선택도 값을 ECQO 결과로 대체
- 4. 물리 계획 생성  
    정확한 선택도를 기반으로 조인 순서/방식 결정

벤치마크: TPC-H 확장 (벡터 필터 추가)  
플랫폼: DuckDB

|          | 기본 DuckDB | Exqutor DuckDB |
|----------|-----------|----------------|
| 평균 속도향상: | 1배        | 8.3배           |
| 최대 속도향상: | 1배        | 37.2배          |
| 최소 속도향상: | 1배        | 1.5배           |

속도향상 분포:

Q3: 3.2배 (세 개 테이블 조인)  
Q5: 5.8배 (지역/공급자 분석)  
Q8: 37.2배 (매우 복잡한 조인) ← 최대  
Q9: 2.1배 (간단한 조인)  
Q10: 9.1배 (네 개 테이블 조인)

· r 2pn ↔ E

· r 2pn ↔ 43324333

J가hvJH? /

D] AS

J가hvJH K감가↔ ?

41

· t ty—two가hvom

51URGV

+ / ,

61

P가 값↔

D] A/

YMS UJ 534>

J가 hvJ H

URGV

1

K감가↔ ?

K감가↔ ?

41· r 2pn ~↔ V ~↔ p YWR ,? XJ HS Y

51aHGYK + J H,? 0 J H

61J가 hvJ H + URGV,?

?

· r 2pn ~↔?# XJ HS Y #

aHGYK?# J H #

J가 hvJ H?# J H#

K감가↔ /

1

41

J가 hvJ H0aYY NTYb

+tyrwp 간↔p↔ ?

ni hs 가 ol p,

51

J가 hvJ H / ?

+ ,

61

?

+ ? ,

NTYb ,

DA

J가 hvJ H K감가↔

/KI WU

?

EA

J가hvJH / ?

JH

I A

J가hvJH0aYY ?  
NTYb +aL/ ,  
+pnl w각 +

---